

CLICK-BY-CLICK WALKTHROUGH · NO PRIOR EXPERIENCE REQUIRED

Build the QuoteMax automation, **one click at a time.**

This is the slowest, most pedantic version of the build guide. Every button, every field, every menu — exactly what to click and what to type. If you have a laptop and a credit card, you can finish this. **Read line by line; don't skip.**

AUDIENCE

First-time builder

TOTAL TIME

~2 working days + 1–3 day Twilio wait

COST TO TEST

~\$20–50 AUD

Before you start, read this once.

You're going to build a phone-answering AI. By the end, a homeowner can dial a number, have a conversation with an AI receptionist, and a draft quote will appear in your database 45 seconds after they hang up. The build is split into **5 stages** from the architecture diagram, plus two **foundation blocks** in between that wire the plumbing.

This guide pairs with two reference docs: [architecture.html](#) shows you the system shape; [build-guide.html](#) has the full code blocks. **This file is the click-by-click order.** Open it in one browser tab and keep building in the other.

Important rules of thumb:

- If a command says "in your terminal" — that means a command-line window. On Windows, search "PowerShell" or "Command Prompt" in the Start menu. On Mac, search "Terminal" in Spotlight.
- If something says **Click this** — that's a real button label or menu item you'll see on screen.

- If something says `Settings → API` — that means open the Settings menu, then click the API submenu.
- If something says `your_value_here` — that's a field you fill in.
- If a step fails, re-read the step. Don't improvise. Most beginner failures are skipped sub-steps.
- **Save every API key the moment you create it.** Use a password manager or a single text file you don't share. Losing keys means starting over for that service.

The full path.

P0a	Pre-flight A — Install tools on your laptop	~30 MIN
P0b	Pre-flight B — Create accounts	~45 MIN
S01	Stage 01 — Customer call origination	READ ONLY
S02	Stage 02 — Provision Twilio AU number	+ 1-3 DAY WAIT
F1	Foundation 1 — Backend skeleton	~2 HR
F2	Foundation 2 — Database setup	~1 HR
S03	Stage 03 — Vapi AI Receptionist	~3 HR
S04	Stage 04 — Intake Engine	~2 HR
S05	Stage 05 — Estimation Engine	~4 HR
V	Verify — End-to-end test	~30 MIN
T	Troubleshooting — common failures	REFERENCE

Install five programs.

These are the tools that let you write code, run code, and connect your laptop to the internet so external services can talk to it. Install all five before going further.

A.1 — Install Node.js

Node.js is the program that runs JavaScript on your computer. Without it, nothing else works.

1 Open your web browser

Go to <https://nodejs.org/>.

2 Pick the LTS version

You'll see two big green download buttons. Click the one labeled **LTS** (it'll say something like "20.x.x LTS" — "LTS" means "Long Term Support" — the stable one). Don't click the "Current" one.

3 Run the installer

Open the file you just downloaded (it'll be in your Downloads folder). Click **Next** through every screen — accept defaults. On the "Tools for Native Modules" screen, leave the checkbox **unchecked** and click Next. Click **Install** on the final screen.

4 Verify it worked

Open a terminal:

- **Windows:** press the Windows key, type `PowerShell`, press Enter.
- **Mac:** press Cmd+Space, type `Terminal`, press Enter.

In the terminal that opens, type:

```
node --version
```

Press Enter. You should see something like `v20.18.0`. If you see "command not found" or "not recognised", close the terminal completely and open a brand new one — Node only appears in terminals opened *after* install.

A.2 — Install pnpm

pnpm is a faster, tidier version of npm (which came with Node). Every "install this library" command will use pnpm.

1

In the same terminal, type

```
npm install -g pnpm
```

Press Enter. Wait ~30 seconds for it to finish.

2

Verify it worked

```
pnpm --version
```

You should see something like `9.x.x`.

A.3 — Install Git

Git tracks every change you make to your code. You'll need it to push code to GitHub later.

1

Download the installer

Go to <https://git-scm.com/>. The download starts automatically for your OS — click `Click here to download` if it doesn't.

2

Run the installer

Click `Next` through every screen. Defaults are fine for everything. On Windows, when asked about default editor, you can leave it as Vim (you won't use it).

3

Configure your name and email

In your terminal:

```
git config --global user.name "Your Real Name"
git config --global user.email "you@example.com"
```

Replace the values with your own. These get stamped on every code commit you make.

4

Verify it worked

```
git --version
```

You should see `git version 2.x.x`.

A.4 — Install VS Code

VS Code is the program where you'll actually write code. It's free and made by Microsoft.

1

Download

Go to <https://code.visualstudio.com/> and click the big blue download button for your OS.

2

Install

Run the installer. **On Windows**, on the "Select Additional Tasks" screen, tick all four checkboxes — especially "Add to PATH" and "Open with Code" — they make life easier later.

3

Open VS Code once

Just open it after install. You don't need to do anything in it yet — we just want it ready.

A.5 — Install ngrok

ngrok creates a temporary public web address that points to your laptop. Twilio and Vapi need a public URL to send webhooks to — your laptop doesn't have one normally, so ngrok bridges the gap.

1

Sign up

Go to <https://ngrok.com/>. Click `Sign up for free` (top right). Use Google or GitHub login if you have one — fastest path.

2

Get your auth token

After signing up, you'll land on the dashboard. In the left sidebar, click `Your Authtoken`. Click the `Copy` button next to the long string.

3

Download the binary

Still on the ngrok dashboard, click `Setup & Installation` in the left sidebar. Pick your OS and follow the download instructions there. On Windows, the simplest path is to

download the ZIP, extract it, and put `ngrok.exe` somewhere easy like `C:\ngrok\`.

4

Add the auth token

Back in your terminal:

```
ngrok config add-authtoken PASTE_YOUR_TOKEN_HERE
```

Replace `PASTE_YOUR_TOKEN_HERE` with the token you copied. You should see `Authtoken saved`.

5

Don't run ngrok yet

You'll start it later in Foundation 1. For now, just having it installed is enough.

DONE CHECK – PRE-FLIGHT A

In a fresh terminal, all four of these print versions: `node --version`, `pnpm --version`, `git --version`, `ngrok --version`. VS Code opens when you launch it. If any one of these fails, fix it before continuing.

PRE-FLIGHT B • SERVICE ACCOUNTS

Sign up for seven services.

Each service does one thing. You'll grab an API key from each one. **Open a fresh text file or password manager and paste each key as you create it** — you'll need them all in Foundation 1.

CARD WARNING

Twilio and Anthropic require a credit card upfront. Set spend limits where possible. Total cost while learning: ~\$20–50. **Never paste API keys into ChatGPT, Slack, or anywhere public.** If you accidentally leak one, immediately revoke it in that service's dashboard.

B.1 — Twilio (phone numbers + SMS)

1

Sign up

Go to <https://www.twilio.com/try-twilio>. Fill in name, email, password. Verify your email + phone via the codes they send.

2

Answer the onboarding questions

Pick: "What do you want to do first?" → **Get a phone number**. "Which language?" → **Node.js**. Skip anything optional.

3

Save your credentials

You'll land on the Twilio Console homepage. On the right side under **Account Info**, you'll see:

- **Account SID** — starts with **AC ...** — copy it, paste into your text file labeled `TWILIO_ACCOUNT_SID`
- **Auth Token** — click the **show** link to reveal it — copy it, paste as `TWILIO_AUTH_TOKEN`

4

Add a credit card

Click your account name (top right) → **Billing → Manage billing** → add card. Twilio gives you ~\$15 free trial credit; you'll spend ~\$1.50/mo on the AU number plus ~2c per minute of inbound calls.

B.2 — Vapi (voice AI orchestrator)

1

Sign up

Go to <https://vapi.ai/>. Click **Sign Up** (top right). Use Google login — it's the fastest.

2

Skip the onboarding wizard

Vapi will try to walk you through creating an assistant. Click **Skip** or close it — we'll do that properly in Stage 03.

3

Get your API key

Left sidebar → **API Keys** (sometimes under "Settings" depending on the dashboard version). Click **Create API Key**. Name it `quotemate-dev`. Copy the key the moment it shows — you can't see it again. Paste as `VAPI_API_KEY`.

4

Note your free credit

You should see ~\$10 of free credit on the dashboard — enough for ~50 test calls.

B.3 — Deepgram (speech-to-text)

1

Sign up

Go to <https://console.deepgram.com/signup>. Use email or Google login.

2

Skip the project naming

If asked, name your project `quotemate`.

3

Create an API key

Left sidebar → `API Keys` → `Create a New API Key`. Name it `quotemate-dev`. **Permissions:** tick `Member`. **Expiration:** No expiration. Click `Create Key`. Copy and paste as `DEEPGRAM_API_KEY`.

4

Note your free credit

\$200 of free credit comes with signup — plenty for testing.

B.4 — ElevenLabs (text-to-speech)

1

Sign up

Go to <https://elevenlabs.io/>. Click `Sign Up` (top right).

2

Get your API key

Click your profile circle (top right) → `Profile + API Key`. Click the eye icon next to "API Key" to reveal it. Copy and paste as `ELEVENLABS_API_KEY`.

3

Free tier

10K characters/month free — enough for testing dozens of calls.

B.5 — Anthropic (Claude AI)

1

Sign up

Go to <https://console.anthropic.com/>. Sign up with Google or email.

2

Verify phone number

Required for fraud prevention. Use a real mobile.

3

Add a payment method

Left sidebar → Plans & Billing → Add payment method. Add a card.

4

Set a spend limit

Same page → set **Monthly spend limit** to \$20 while you're learning. This is your safety net — you can raise it later.

5

Create an API key

Left sidebar → API Keys → Create Key. Name it quotemate-dev. Workspace: Default. Click Create. Copy the key (starts with sk-ant- ...) and paste as ANTHROPIC_API_KEY.

B.6 — Supabase (database)

1

Sign up

Go to <https://supabase.com/dashboard/sign-up>. Click Continue with GitHub (if you don't have GitHub yet, create it at github.com/signup first — you'll need it for Vercel anyway).

2

Don't create a project yet

You'll do that in Foundation 2 — for now we just need the account to exist.

B.7 — Vercel (hosting)

1

Sign up

Go to <https://vercel.com/signup>. Click Continue with GitHub.

2

Pick the Hobby (free) tier

When asked about plans, pick Hobby. Free tier covers everything in this build.

DONE CHECK — PRE-FLIGHT B

Your text file should now have these 7 keys saved (Supabase keys come later in Foundation 2):

```
TWILIO_ACCOUNT_SID=AC ...  
TWILIO_AUTH_TOKEN= ...  
VAPI_API_KEY= ...  
DEEPGRAM_API_KEY= ...  
ELEVENLABS_API_KEY= ...  
ANTHROPIC_API_KEY=sk-ant- ...
```

Plus a working Supabase + Vercel + GitHub login. If any account didn't accept your card or signup, get that working before continuing.

STAGE 01 · CUSTOMER CALL ORIGINATION

Nothing to install — read this and move on.

Stage 01 in the architecture is just "a homeowner picks up their phone and dials a number." There's no setup. The whole point of Stages 02 → 05 is to make sure that *something good happens* when they do. Move on.

STAGE 02 · PROVISION YOUR AU PHONE NUMBER

Buy a real Australian phone number.

This is the number a homeowner will dial. Australia requires identity verification before you can buy one — that part takes 1–3 business days. **Start this step early** while the rest of the build progresses, because you're blocked on Twilio's regulator review.

S2.1 — Submit the regulatory bundle

1

Go to the Twilio Console

Open <https://console.twilio.com/>. Make sure you're in the project you signed up with.

2

Find Regulatory Compliance

In the left sidebar, click `Phone Numbers` to expand it, then

`Regulatory Compliance → Bundles`. (If "Phone Numbers" isn't in the sidebar, click the

`Explore Products` button at the bottom of the sidebar and find it there.)

3

Create a new bundle

Click the `Create new bundle` button (top right).

4

Pick country and number type

- Country: `Australia`
- Number type: `Local` (not Mobile, not Toll-free)
- End user type: `Individual` (or `Business` if you have an ABN — Business gets faster approval)

Click `Next`.

5

Fill in your details

Name, address, phone number, date of birth. **Use a real Australian address** — they verify it.

6

Upload your ID

Photo of your driver's licence or passport. Make sure all corners are visible and text is readable.

7

Submit

Click `Submit for review` at the bottom. The status will show **"In review"**.

WAIT TIME — START THE REST OF THE BUILD NOW

Approval takes **1–3 business days**. While you wait, you can do [Foundation 1](#), [Foundation 2](#), and even start configuring Stage 03 in Vapi. Don't sit and refresh the Twilio page — go build.

S2.2 — Once approved: buy your number

You'll get an email when the bundle is approved. Then:

1 Go to the buy-a-number page

Twilio Console → `Phone Numbers → Manage → Buy a number`.

2 Filter for Australia

- Country: `Australia`
- Capabilities: tick `Voice` only (you don't need SMS or MMS for v1)
- Number Type: `Local`

Click `Search`.

3 Pick a Sydney or Melbourne number

You'll see a list. Pick one with a +61 2 (Sydney) or +61 3 (Melbourne) prefix — local numbers feel more trustworthy than 1300s. Click the `Buy` button next to it.

4 Confirm

Confirmation popup → tick the box that says you'll comply with the bundle → `Buy`. Cost is around AU\$1.50/month.

5 Save the number

The number now appears in `Phone Numbers → Manage → Active numbers`. Copy it (in `+61 ...` format with no spaces). Paste in your text file as `TWILIO_PHONE_NUMBER`.

S2.3 — Test it rings

1 Dial your new number

From your mobile, dial it. After a couple of rings, you'll hear Twilio's default voice say something like *"Hello, this is your Twilio number."*

2 Hang up

That's all we need for now. In Stage 03 we'll point this number at the AI receptionist.

DONE CHECK — STAGE 02

You have a real `+61` phone number that rings when you dial it. The Account SID, Auth Token, and Phone Number are all saved in your text file.

Create your Next.js project.

This is where all your code will live. The voice agent and the two AI engines need somewhere to receive HTTP requests — a Next.js app deployed on Vercel is the cheapest, simplest way to host them.

F1.1 — Generate the project

1

Open a terminal

(See Pre-flight A.1 if you forgot how.)

2

Navigate to where you want the project

For example, on Windows:

```
cd C:\Users\YourName\Desktop
```

On Mac:

```
cd ~/Desktop
```

3

Run the create-next-app command

```
pnpm create next-app@latest quotemate-automation
```

You'll be asked a series of questions. Answer them **exactly like this**:

- "Would you like to use TypeScript?" →
- "Would you like to use ESLint?" →
- "Would you like to use Tailwind CSS?" →

- "Would you like your code inside a `src/` directory?" →
- "Would you like to use App Router?" →
- "Would you like to use Turbopack for `next dev`?" →
- "Would you like to customize the import alias?" →

Wait ~2 minutes for it to install everything.

F1.2 — Open it in VS Code and verify it runs

1

Move into the project folder

```
cd quotemate-automation
```

2

Open it in VS Code

```
code .
```

(That's "code" + space + period.) VS Code will open with the project. If `code` isn't recognised, open VS Code manually and use `File → Open Folder` to pick the `quotemate-automation` folder.

3

Start the dev server

Back in your terminal:

```
pnpm dev
```

You should see "Ready in X.Xs" and "Local: http://localhost:3000".

4

Open it in your browser

Go to <http://localhost:3000>. You should see the default Next.js welcome page. If it loads, your backend is alive.

5

Stop the server

In the terminal, press `Ctrl+C` (or `Cmd+C` on Mac) to stop it. We'll restart it later.

F1.3 — Install the libraries you'll need

1

In your terminal, type

```
pnpm add ai @ai-sdk/anthropic zod twilio @supabase/supabase-js
```

This adds the Vercel AI SDK, the Anthropic provider, Zod (schema validation), Twilio's SDK, and the Supabase client. Wait ~30 seconds.

F1.4 — Create your env file

1

In VS Code, create a new file at the project root

Right-click in the file explorer (left sidebar) → **New File** → name it exactly `.env.local` (the leading dot matters).

2

Paste this template

```
# Twilio
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
TWILIO_PHONE_NUMBER=+61...

# Vapi
VAPI_API_KEY=...
VAPI_WEBHOOK_SECRET=

# Voice services
DEEPGRAM_API_KEY=...
ELEVENLABS_API_KEY=...

# Claude
ANTHROPIC_API_KEY=sk-ant-...

# Supabase (filled in F2)
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# App URL — for fire-and-forget handoffs between API routes
APP_URL=http://localhost:3000
```

3

Replace each value

Replace each `...` with the actual key from your text file. Leave the Supabase values blank — you'll fill those in Foundation 2. Leave `VAPI_WEBHOOK_SECRET` blank for now.

4

Save the file

Ctrl+S (or Cmd+S).

DON'T SHARE .ENV.LOCAL

Git ignores `.env.local` by default. Never paste its contents anywhere. If you accidentally publish a key, immediately revoke it in that service's dashboard and create a new one.

F1.5 — Push to GitHub

1

Initialise the git repo

In the terminal (still in the project folder):

```
git init
git add .
git commit -m "Initial automation backend skeleton"
```

2

Create the GitHub repo

Go to <https://github.com/new>.

- Repository name: `quotemate-automation`
- Description: optional
- Visibility: `Private` (don't make it public — even though no secrets are committed, you don't need it indexed by Google)
- **Don't tick** "Add a README", ".gitignore", or "license" — your local repo already has the structure

Click `Create repository`.

3

Push your code

GitHub will show you setup instructions. Copy the three lines under "push an existing repository" — they'll look like:


```
git remote add origin https://github.com/YOUR_USERNAME/quotemate-automation.git
git branch -M main
git push -u origin main
```

Paste them into your terminal. If asked for a username/password, use your GitHub username and a **Personal Access Token** (not your password) — create one at github.com/settings/tokens if needed.

F1.6 — Deploy to Vercel

1 Import the repo

Go to <https://vercel.com/new>. You'll see a list of your GitHub repos. Click **Import** next to `quotemate-automation`.

2 Configure the project

Vercel auto-detects it's a Next.js app. Don't change anything. Click **Deploy**.

3 Wait ~1 minute

Vercel builds and deploys. When it's done, you'll see a confetti animation and your live URL (something like `quotemate-automation-abc123.vercel.app`).

4 Add env vars to Vercel

This is critical — without them, the deployed version can't talk to anything. Click your project name → `Settings → Environment Variables`. For each line in your

`.env.local`:

- Name: paste the variable name (e.g. `TWILIO_ACCOUNT_SID`)
- Value: paste the value
- Environments: tick all three (Production, Preview, Development)
- Click **Save**

Repeat for every variable. **Don't skip any.** You'll need to come back and add the Supabase ones in F2.

5 Update APP_URL in Vercel

Set `APP_URL` in Vercel to your live Vercel URL (e.g. `https://quotemate-automation-abc123.vercel.app`). In your local `.env.local`, leave it as `http://localhost:3000` for now.

F1.7 — Start ngrok

For local development, Twilio and Vapi need a public URL pointing to your laptop. ngrok provides that.

1 Open a **SECOND** terminal

Don't close the first one (where you ran `pnpm dev`). Open a second one. On Windows, you can right-click the taskbar icon and click "PowerShell".

2 Start ngrok

```
ngrok http 3000
```

You'll see a screen with a "Forwarding" line like:

```
Forwarding   https://abc123.ngrok.app → http://localhost:3000
```

3 Copy the https URL

The `https://abc123.ngrok.app` part. Paste it somewhere — you'll use it in Stage 03.

4 Leave ngrok running

Don't close this terminal. If you close it, the URL changes and you'll have to re-paste it into Vapi.

DONE CHECK — FOUNDATION 1

(1) `pnpm dev` runs in terminal 1, localhost:3000 shows the welcome page. (2) `ngrok http 3000` runs in terminal 2 with a public URL. (3) Your project is on GitHub. (4) Vercel shows a successful deployment. (5) All env vars are pasted into both `.env.local` and Vercel.

All seven pipeline tables, the pgvector extension, the similarity-search function, and the seeded "easy 5" assemblies. After this, the rest of the build just inserts and reads from this database.

F2.1 — Create the project

1 Go to Supabase dashboard

<https://supabase.com/dashboard> and log in.

2 Click New Project

Top right of the dashboard. (If it's your first project, you'll need to create an Organization first — name it whatever you want, e.g. your name.)

3 Fill in the project form

- Name: `quotemate-automation-dev`
- Database Password: **generate a strong one** using the **Generate a password** button. Copy it and save in your text file as `SUPABASE_DB_PASSWORD`. You probably won't use it directly, but losing it means you can't reset things.
- Region: pick `Sydney (ap-southeast-2)` for AU latency, or whichever is closest if you're elsewhere.
- Pricing Plan: `Free` is fine for development.

Click **Create new project**.

4 Wait ~2 minutes

Supabase provisions the database. The dashboard will show "Setting up project..." with a progress indicator.

F2.2 — Copy your URL and keys

1 Find the API page

Once the project is ready, click `Project Settings` (gear icon at the bottom of the left sidebar) → `API`.

2 Copy three values

- **Project URL** (under "Project URL") — paste in `.env.local` as `NEXT_PUBLIC_SUPABASE_URL`
- **anon public** (under "Project API keys") — click **Reveal**, copy, paste as `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- **service_role** (under "Project API keys", below anon) — click **Reveal**, copy, paste as `SUPABASE_SERVICE_ROLE_KEY`

Save your `.env.local`.

3

Add the same three to Vercel

Vercel project → `Settings → Environment Variables`. Same three names and values. Tick all three environments. Save each.

SERVICE-ROLE KEY IS SENSITIVE

The `service_role` key bypasses Row-Level Security. Use it only in server-side code, **never in browser code**. The "NEXT_PUBLIC_" prefix in `NEXT_PUBLIC_SUPABASE_ANON_KEY` means that key gets exposed to the browser — that's fine for the anon key but never do that for `service_role`.

F2.3 — Enable the pgvector extension

1

Open the Database menu

In the left sidebar of your Supabase project, click `Database` (the icon looks like a cylinder).

2

Click Extensions

In the submenu that appears, click `Extensions`.

3

Find "vector"

Use the search box at the top — type `vector`. You'll see `pgvector` in the list (it might be labeled `vector`).

4

Toggle it on

Click the toggle switch on the right side of the row. A confirmation dialog might appear — click **Enable extension**.

5

Verify

The toggle should now be green/on, and the row should show "Enabled".

F2.4 — Create the library tables

1

Open the SQL Editor

Left sidebar → **SQL Editor** . Click **New query** if a blank editor isn't already open.

2

Paste this SQL

Clear the editor (Ctrl+A, Delete) and paste:

```
create table shared_assemblies (  
  id uuid primary key default gen_random_uuid(),  
  trade text not null default 'electrical',  
  name text not null,  
  description text,  
  default_unit text,  
  default_unit_price_ex_gst numeric(10,2),  
  default_labour_hours numeric(6,2),  
  default_exclusions text  
);  
  
create table shared_materials (  
  id uuid primary key default gen_random_uuid(),  
  trade text not null default 'electrical',  
  name text not null,  
  brand text,  
  unit text,  
  default_unit_price_ex_gst numeric(10,2)  
);  
  
create table pricing_book (  
  id uuid primary key default gen_random_uuid(),  
  hourly_rate numeric(8,2) default 110,  
  call_out_minimum numeric(8,2) default 150,  
  apprentice_rate numeric(8,2) default 60,  
  default_markup_pct numeric(5,2) default 28,  
  risk_buffer_pct numeric(5,2) default 15,  
  gst_registered boolean default true,  
  licence_type text,  
  licence_number text,  
  licence_state text,  
  licence_expiry date,  
  overlays jsonb default '{}'::jsonb  
);
```

3

Run it

Click the green **Run** button (bottom right) or press Ctrl+Enter. You should see "Success. No rows returned." at the bottom.

F2.5 — Create the pipeline tables

1

Click "+" to make a new query

(or clear the editor again).

2

Paste the full pipeline schema

This is the long block from [build-guide.html step 5](#) — copy it from there starting at `create table calls` and ending at the closing semicolon of `quote_line_items`. It defines: `calls`, `intakes` (with the embedding column), `quotes` (with good/better/best JSONB), and `quote_line_items`.

3

Run it

Click **Run**. "Success. No rows returned."

F2.6 — Create the similarity-search function

1

New query, paste this

```
create or replace function match_intakes(
  query_embedding vector(1536),
  match_count int default 5
)
returns table (id uuid, scope jsonb, similarity float)
language sql stable as $$
  select id, scope, 1 - (embedding ⇔ query_embedding) as similarity
  from intakes
  where embedding is not null
  order by embedding ⇔ query_embedding
  limit match_count;
$$;
```

(Note: the `⇔` in the SQL renders as `⇔` — three characters — when you paste it. That's pgvector's cosine-distance operator.)

2

Run it

"Success. No rows returned." This function lets Stage 04 find the 5 most similar past intakes given a new one's vector.

F2.7 — Seed the "easy 5" + a pricing book row

1 New query, paste the seed inserts

Copy the full seed block from [build-guide.html step 5](#) (starts with `insert into shared_assemblies`, includes 5 assemblies + 8 materials + 1 pricing_book row).

2 Run it

You should see "Success. Rows: 5" or similar — different counts for each insert statement. Some Supabase versions show only the last result; that's fine.

3 Verify the seeds landed

Left sidebar → `Table Editor` → click `shared_assemblies`. You should see 5 rows: "Install LED downlight", "Replace double GPO", "Install customer-supplied ceiling fan", "Hardwire 240V smoke alarm", "Install outdoor IP-rated LED light". Click `shared_materials` — 8 rows. Click `pricing_book` — 1 row with `hourly_rate = 110`.

DONE CHECK — FOUNDATION 2

Table Editor shows all 7 tables: `shared_assemblies` (5 rows), `shared_materials` (8 rows), `pricing_book` (1 row), `calls` (0 rows), `intakes` (0 rows), `quotes` (0 rows), `quote_line_items` (0 rows). Database → Extensions shows pgvector enabled.

STAGE 03 • VAPI AI RECEPTIONIST

Build the AI that answers the phone.

Connect Twilio to Vapi, configure the voice assistant, paste in the system prompt that drives the conversation, then build the webhook receiver in your Next.js app. By the end of this stage you'll be able to dial your number and have a real conversation with the AI.

S3.1 — Connect Twilio to Vapi

1 Open the Vapi dashboard

<https://dashboard.vapi.ai/>.

2 Go to Phone Numbers

Left sidebar → Phone Numbers .

3 Import from Twilio

Click the **Import Number** or **+** button → choose **Twilio** from the provider list.

4 Paste credentials

Fill in:

- Twilio Account SID: `your AC...`
- Twilio Auth Token: `your auth token`
- Twilio Phone Number: `+61... (no spaces)`

Click **Import from Twilio**. Vapi automatically updates Twilio's voice webhook to point to Vapi — you don't need to touch Twilio for this.

5 Verify

The number should now appear in your Vapi **Phone Numbers** list.

S3.2 — Plug in your provider keys (optional but recommended)

By default Vapi uses its own Deepgram, ElevenLabs, and Anthropic keys and bills you with a markup. Using your own keys means retail rates.

1 Go to Provider Keys

Left sidebar → **Settings → Provider Keys** (or **Org → Provider Keys**).

2 Paste each key

Add Deepgram, ElevenLabs, and Anthropic keys from your text file. Save each.

S3.3 — Create the assistant

1 Go to Assistants

Left sidebar → Assistants → Create Assistant (top right).

2

Name it

`QuoteMax Receptionist`. Click **Create**.

3

Configure the Transcriber tab

- Provider: `Deepgram`
- Model: `nova-2`
- Language: `en-AU` (or "English (Australia)")

4

Configure the Voice tab

- Provider: `ElevenLabs`
- Voice: pick an Australian-sounding voice. Vapi has previews — click the play button to hear samples. Pick one that sounds friendly and natural. Save the voice ID it shows.

5

Configure the Model tab

- Provider: `Anthropic`
- Model: `claude-haiku-4-5` (or the newest Haiku available — Haiku is fast and cheap, perfect for the live conversation)
- Max tokens: leave default (~250)
- Temperature: `0.4` (low temp = consistent question routing)

6

Set the First Message

This is what the AI says when the call connects. Paste:

```
G'day, you've reached the AI quoting line for [your business name]. I can take de
```

The "may be recorded" line is required for AU consent compliance.

S3.4 — Paste the system prompt

1

Find the System Prompt field

Usually under the Model tab, scroll to "System Prompt" or "System Message".

2

Open the build guide in another tab

Open [build-guide.html step 6](#). Find the long code block that starts with `ROLE` and ends with `- Skip photo asks for switchboard / EV / outdoor / oven jobs`.

3

Copy the entire prompt

Don't trim it — every section drives a piece of the conversation logic.

4

Paste it into the System Prompt field in Vapi

Click `Save` (or it auto-saves).

WHY THE PROMPT IS SO LONG

It encodes 9 different job-flow question trees, the emergency override logic, the photo capture protocol, and the confidence scoring rules. Each section drives an actual decision the AI will make on the call. Trimming it = breaking the pipeline.

S3.5 — Set the Server URL (the webhook target)

1

Find Server URL in the assistant config

Usually in an "Advanced" or "Server" section of the assistant settings.

2

Paste your ngrok URL + the path

If your ngrok URL is `https://abc123.ngrok.app`, paste:

```
https://abc123.ngrok.app/api/vapi/webhook
```

Important: use the ngrok URL, not localhost. Vapi's servers can't reach your laptop directly.

3

Save

If there's a separate Save button for this section, click it.

S3.6 — Connect the assistant to your phone number

1

Go back to Phone Numbers

Left sidebar → Phone Numbers .

2

Click your AU number

The row with your +61... number.

3

Set the Inbound Assistant

In the "Inbound Settings" section, set "Assistant" to QuoteMax Receptionist . Save.

S3.7 — Build the webhook receiver in your Next.js project

1

Create the folder structure in VS Code

In the file explorer, right-click on the app folder → New Folder → name it api . Inside api , create another folder called vapi . Inside vapi , create another called webhook . Inside webhook , create a file called route.ts .

2

Paste the webhook code

Open [build-guide.html step 6](#) and find the code block that starts with import { createClient } from '@supabase/supabase-js' . Copy the whole block and paste it into app/api/vapi/webhook/route.ts . Save.

3

Restart pnpm dev

If pnpm dev isn't already running in terminal 1, start it: pnpm dev . If it was running, leave it — Next.js auto-reloads on file changes.

S3.8 — Test the call

1

Verify both terminals are running

Terminal 1: pnpm dev shows "Ready". Terminal 2: ngrok http 3000 shows the public URL.

2

Dial your AU number from your mobile

Speak naturally: "Hi, I need six LED downlights installed in my kitchen, replacing the old halogens. Wiring's already there. Single-storey house, plaster ceiling."

3

Let the conversation flow

The AI should ask follow-up questions matching the system prompt's downlights flow. End the call when it summarises back.

4

Check Supabase

Supabase → Table Editor → `calls`. There should be a new row with the transcript and `recording_url` populated. Open it; read the transcript to make sure the AI captured the right thing.

Troubleshooting: If no row appears in `calls`, check (a) ngrok is still running with the same URL you pasted into Vapi, (b) the Server URL in Vapi includes `/api/vapi/webhook` at the end, (c) the route file is at `app/api/vapi/webhook/route.ts` exactly. Check Vapi's `Logs` to see if it tried to call your webhook and got an error.

DONE CHECK — STAGE 03

You can dial your AU number, have a conversation with the AI, hang up, and ~10 seconds later see a row in `calls` with the full transcript.

STAGE 04 • INTAKE ENGINE

Turn the transcript into structured data.

Free-form speech is useless to the Estimator. The Intake Engine reads the transcript and produces strict JSON: `job_type`, `scope`, `risks`, `confidence`. Three small files plus one API route.

S4.1 — Create the intake schema

1

Create the folder

In VS Code, at the project root, create folder `lib` → inside it, create folder `intake`.

2

Create schema.ts

Inside `lib/intake`, create file `schema.ts`.

3

Paste the IntakeSchema

From [build-guide.html step 7](#), copy the full `IntakeSchema` block (starts with `import { z } from 'zod'`, ends with `export type Intake = z.infer<typeof IntakeSchema>`).

4

Save

Ctrl+S.

S4.2 — Create the structuring function

1

Create structure.ts

Inside `lib/intake`, create file `structure.ts`.

2

Paste the structureIntake function

From [build-guide.html step 7](#), copy the full block that starts `import { anthropic } from '@ai-sdk/anthropic'`.

3

Save

S4.3 — Create the embedding helper

1

Create embed.ts

Inside `lib/intake`, create file `embed.ts`.

2

Paste the embedIntake function

From [build-guide.html step 7](#), copy the embed block.

3

Voyage vs OpenAI

If you'd prefer to use OpenAI for embeddings instead of Voyage, swap the model line per the note at the bottom of step 7. Otherwise leave it as voyage-3.

S4.4 — Build the API route

1

Create the folder structure

Inside `app/api`, create folder `intake`, then inside `intake` create folder `structure`, then inside `structure` create file `route.ts`.

2

Paste the route code

From [build-guide.html step 7](#), copy the route handler that loads the call, structures it, embeds it, saves it, and fires `/api/estimate/draft`.

3

Save and let Next.js reload

The terminal running `pnpm dev` will recompile.

S4.5 — Test it

1

Make another test call

Same way as Stage 03 — dial your number, describe a job, hang up.

2

Check the intakes table

Supabase → Table Editor → `intakes`. Within ~20 seconds of hanging up, a row should appear with:

- `call_id` linked to the call you just made
- `job_type` populated (e.g. "downlights")
- `scope` JSONB with `item_count`, `indoor_outdoor`, etc.
- `risks` array (might be empty for clean jobs)
- `embedding` shows as a long array of numbers — that's the 1536-dim vector
- `confidence` = HIGH / MEDIUM / LOW

3

Sanity check

Did the AI extract the job correctly? If you said "six downlights", does `scope.item_count` = 6? If not, the system prompt might need tightening.

DONE CHECK — STAGE 04

One test call produces one row in `calls` AND one row in `intakes` with structured fields and a non-null embedding.

Draft the actual quote.

Claude Opus reads the structured intake, calls four tools to look up assemblies, materials, and apply markup, then writes Good / Better / Best pricing tiers. This is the most code, but every piece maps to a box in the Stage 05 cluster of the architecture diagram.

S5.1 — Create the tools file

1

Create the folder

Inside `lib`, create folder `estimate`.

2

Create tools.ts

Inside `lib/estimate`, create file `tools.ts`.

3

Paste the four tools

From [build-guide.html step 8](#), copy the block that defines `lookupAssembly`, `lookupMaterial`, `applyMarkup`, `flagInspectionNeeded`.

4

Save

S5.2 — Create the system prompt

1

Create prompt.ts

Inside `lib/estimate`, create file `prompt.ts`.

2

Paste the systemPrompt function

From [build-guide.html step 8](#), this is a long block (~200 lines) that defines the entire Estimator's behaviour — Good/Better/Best framing, calculation order, risk-buffer triggers, inspection fallback, fault-finding override. **Copy all of it.**

3

Save

S5.3 — Create the runner

1 Create run.ts

Inside `lib/estimate`, create file `run.ts`.

2 Paste the runEstimation function

From [build-guide.html step 8](#), copy the runner block. The key part is `generateText({ ..., tools, maxSteps: 10 })` — that's what gives Claude the agency to call tools in a loop.

3 Save

S5.4 — Build the API route

1 Create folder structure

Inside `app/api`, create folder `estimate`, inside that create `draft`, inside that create file `route.ts`.

2 Paste the route code

From [build-guide.html step 8](#), copy the full route handler that loads the intake, loads the pricing book, runs the estimation, computes GST, and inserts into `quotes`.

3 Save and let Next.js reload

S5.5 — Test it

1 Make another test call

Describe a clean "easy 5" job: "I need six downlights in my kitchen, replacing existing halogens, wiring's already there, plaster ceiling, indoor."

2 Wait ~45 seconds total

Vapi webhook → calls (10s) → intakes (20s) → quotes (45s).

3 Check the quotes table

Supabase → Table Editor → `quotes`. New row should have:

- `intake_id` linked correctly

- `scope_of_works` — plain English description
- `good`, `better`, `best` — JSONB columns each containing a label, `line_items` array, `subtotal_ex_gst`, `timeframe`
- `total_inc_gst` — sensible number (for 6 downlights, somewhere around \$400–600 typically)
- `status` = `'draft'`

4

Open the JSONB

Click on the `better` column to see the JSON. You should see real line items like:

```
{
  "label": "Tri-colour LED downlights",
  "line_items": [
    { "description": "Install LED downlight (tri-colour)", "quantity": 6, ... },
    { "description": "Labour", "quantity": 2.4, "unit": "hr", ... }
  ],
  "subtotal_ex_gst": 480.50,
  "timeframe": "Same day"
}
```

If you see this, your full pipeline works.

DONE CHECK – STAGE 05

One test call → 45 seconds later → one row in `quotes` with three real pricing tiers, sensible totals, and `status: 'draft'`.

VERIFY • END-TO-END TEST

Prove the whole thing works.

Two test calls — one easy, one deliberately complex — should exercise both the auto-quote path and the inspection-required path.

V.1 — Pre-flight checklist

- ☐ Terminal 1: `pnpm dev` running with no errors
- ☐ Terminal 2: `ngrok http 3000` running, URL pasted in Vapi Server URL
- ☐ Vapi: assistant connected to your AU number
- ☐ Supabase: `shared_assemblies` has 5 rows, `pricing_book` has 1 row
- ☐ `.env.local` has all keys (no blanks except `VAPI_WEBHOOK_SECRET`)

V.2 — Easy test (auto-quote path)

Dial your AU number. Say:

"Hi, my name's Anant, I need six LED downlights installed in the kitchen, replacing some old halogens. The wiring's already there. It's a single-storey place in Bondi, plaster ceiling, roof access is fine. Tri-colour would be good."

Expected:

- Within 10s: row in `calls` with the transcript
- Within 20s: row in `intakes` with `job_type: 'downlights'`, `scope.item_count: 6`, `confidence: 'HIGH'`, `inspection_required: false`
- Within 45s: row in `quotes` with three real tiers and a total around \$400–700

V.3 — Complex test (inspection path)

Dial again. Say:

"Hey, there's a burning smell coming from my switchboard, and the breakers keep tripping. I also want to add an EV charger. It's an old place — the switchboard still has ceramic fuses."

Expected:

- `intakes.risks` array contains "burning smell", "tripping breakers", "ceramic-fuse switchboard", "EV charger on old board"
- `confidence: 'LOW'`
- `inspection_required: true`
- `timing.urgency: 'emergency'`
- `quotes.needs_inspection: true`
- `quotes` uses indicative ranges, not fixed line items

DONE CHECK — PIPELINE VERIFIED

If both tests produce the expected outcomes, your full Stages 01 → 05 pipeline is working. You've built the QuoteMax automation.

TROUBLESHOOT · COMMON FAILURES

When something breaks.

90% of beginner failures fall into one of these buckets. Check here before assuming the build is broken.

"Command not recognised" after install

Close **every** terminal window and open a brand new one. Programs only appear in terminals opened *after* install.

The AI answers but no row appears in `calls`

Vapi can't reach your webhook. Check:

- ngrok still running with the same URL you pasted into Vapi (it changes if ngrok restarts)
- Vapi Server URL ends in `/api/vapi/webhook`
- The file is at `app/api/vapi/webhook/route.ts` exactly (case-sensitive)
- Vapi `Logs` shows what it tried — copy any error and search it

Row appears in `calls` but nothing in `intakes`

The intake API route is failing. Check the `pnpm dev` terminal — Next.js prints errors there. Common causes:

- ANTHROPIC_API_KEY missing or wrong in `.env.local`
- Spend limit hit on Anthropic console

- Schema mismatch — Sonnet returned an unexpected field. Loosen the Zod schema or add the field

Row in `intakes` but nothing in `quotes`

The Estimation Engine is failing. Likely:

- Tools returned empty (your seeded library is too sparse — Opus called `flag_inspection_needed` instead). For testing, use the seeded "easy 5" job types.
- Opus output was malformed JSON. Check the dev-server logs for parse errors. Switch from `generateText` to `generateObject` with a Zod schema if it's persistent.
- `maxSteps: 10` exhausted — Opus tried more than 10 tool calls. Either increase the limit or simplify the prompt.

GST math looks wrong (everything off by 10%)

Line items are stored ex-GST; totals are inc-GST. If the customer-visible total is missing the 10%, check the API route's GST calculation — it should be `+(selectedSubtotal * 0.10).toFixed(2)`.

ngrok URL keeps changing

Free ngrok gives you a new random URL each restart. Either:

- Don't restart ngrok during development — leave it running all day
- Upgrade ngrok to a paid plan (\$10/mo) for a static URL
- Or skip ngrok and deploy to Vercel preview branches for each test (slower iteration)